Software Requirements Specification

for

<Code Judge>

Version 1.0 approved

Prepared by TEAM DEVGRU

2nd year ,Btech(H)-SOCSE

RV UNIVERSITY

05-02-2026

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 3
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Intended Audience and Reading Suggestions 4
1.4 Project Scope 6
1.5 References
2. Overall Description 8
2.1 Product Perspective 8
2.2 Product Features 9
2.3 User Classes and Characteristics 10
2.4 Operating Environment 12
2.5 Design and Implementation Constraints 15
2.6 User Documentation 16
2.7 Assumptions and Dependencies 17
3. System Features and Detailed requirements 18
3.1 Functional and Non-functional requirements 18
4. External Interface Requirements 28
4.1 User Interfaces 28
4.2 Hardware Interfaces 30
4.3 Software Interfaces 32
4.4 Communications Interfaces 34
5. Other Requirements
Appendix A: Glossary 36
Appendix B: Analysis Models 38
Appendix C: Issues List 38

Revision History

Name	Date	Reason For Changes	Version
Team Devgru	6/2/2026	Prepared a draft SRS	0.1

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the Code-Judge: Automated Code Evaluation Platform (Release 1.0). This document covers the complete scope of the software system, including the web-based user interface, the microservices backend architecture, and the containerized execution engine.

Code-Judge is designed to address the inefficiencies and security risks inherent in manual programming evaluation within academic labs and coding competitions. Specifically, this platform serves to:

1. **Automate Assessment**: Replace manual logic checking with an automated grading system that compares user code outputs against hidden test cases.
2. **Ensure Environment Consistency**: Eliminate the "it works on my machine" dependency issue by compiling and running code in standardized, isolated environments.
3. **Secure Execution**: Implement a "Sandboxed" execution model using Docker containers to safely run untrusted user code without risking the integrity of the host server.
4. **Streamline Delivery**: Integrate DevOps best practices (CI/CD pipelines) to ensure rapid, reliable deployment and scalability during high-traffic scenarios.

This SRS is intended for the development team (including DevOps, Backend, and Frontend engineers) and project stakeholders to strictly guide the implementation, testing, and deployment of the platform.

1.2 Document Conventions

This document follows standard technical writing practices to ensure clarity and consistency. The following conventions are used throughout this SRS:

Typographical Conventions

- Boldface: Used for section headings, key terms, and significant emphasis (e.g., Docker, Microservices).
- *Italics*: Used for emphasis on specific states (e.g., *Time Limit Exceeded*), variable names, or when introducing new terminology.
- Monospace: Used to denote code snippets, API endpoints (e.g., POST /submit), file paths (e.g., .github/workflows/main.yml), database columns, and system commands.

Requirement Priority Standards Requirements are prioritized using the MoSCoW method or standard "Must/Should/May" terminology to indicate importance:

- MUST: Indicates a mandatory requirement that is critical for the system's core functionality (Release 1.0). The system is not viable without these features (e.g., "The system must isolate code in Docker containers").
- SHOULD: Indicates a high-priority requirement that is highly desirable but not strictly critical for the initial release. These will be implemented if time permits.
- MAY: Indicates a nice-to-have feature or future extension (e.g., "Plagiarism Detection").

Inheritance of Priorities Unless explicitly stated otherwise, lower-level detailed requirements inherit the priority of their parent high-level requirement. For example, if "Security & Isolation" is a MUST, then all specific sub-requirements regarding "Resource Throttling" and "Network Blocking" are also considered mandatory.

1.3 Intended Audience and Reading Suggestions

This Software Requirements Specification is primarily intended for the project development team and stakeholders. The document is organized to provide a high-level overview before diving into specific technical and architectural details.

Intended Audience

1. Backend & Frontend Developers: To understand the functional requirements of the microservices (Auth, Problem, Execution) and the UI interactions (Editor, Dashboard) necessary to build the application.

2. DevOps Engineers & Architects: To find specifications regarding the containerization strategy (Docker), orchestration (Kubernetes/Swarm), and the CI/CD pipeline requirements ensuring system scalability and isolation.
3. Quality Assurance (QA) & Testers: To identify the specific constraints, edge cases (e.g., Time Limit Exceeded), and expected outputs needed to design test plans and validate the system.
4. Project Supervisors (Stakeholders): To verify that the proposed solution meets the security, and plagiarism prevention.

Document Organization

- **Section 2** (Overall Description): Describes the general factors that affect the product and its requirements, including user characteristics and product perspective.
- **Section 3** (System Features): Detailed functional requirements organized by module (User Management, Execution Engine, etc.).
- **Section 4** (External Interface Requirements): Specifications for user interfaces, hardware interfaces, and communication protocols.
- **Section 5** (Non-Functional Requirements): Performance, security, and reliability constraints (e.g., Sandboxing, Scalability).

Suggested Reading Sequence

- For Developers (Backend/Frontend): Begin with Section 2 for context, then focus heavily on Section 3 (Functional Requirements) and Section 4 (Interfaces) to understand the data flow and API contracts.
- For DevOps Engineers: After reviewing Section 2, proceed directly to Section 5 (Non-Functional Requirements) and the System Architecture diagrams to understand the infrastructure, security, and deployment constraints.
- For QA/Testers: Read Section 3 and Section 5 in detail to understand the "Happy Path" and failure states (e.g., TLE, MLE) that must be tested.
- For Stakeholders: Focus on Section 1 and Section 2 to understand the scope and limitations, and review the high-level features in Section 3.

1.4 Project Scope

The software being specified is **Code-Judge**, a web-based, automated code evaluation platform built upon a microservices architecture. It functions as a centralized hub where users can practice coding problems and receive instant, automated feedback. The core of the system relies on a "Sandboxed Execution Engine" utilizing Docker containers to execute untrusted user code securely and efficiently.

Primary Objectives and Goals The primary goal of the Code-Judge platform is to eliminate the bottlenecks associated with manual code evaluation in academic environments. The specific objectives include:

- **Automation of Assessment:** To reduce the turnaround time for grading from days to seconds by automatically validating code against hidden test cases.
- **Environment Standardization:** To resolve "dependency hell" and local environment disparities by executing all submissions in identical, isolated Docker containers.
- **System Scalability:** To utilize Kubernetes and Microservices to support concurrent code submissions from hundreds of users simultaneously.
- **Security Assurance:** To prevent malicious code execution on servers by enforcing strict resource limits and network isolation within the execution sandbox.

Strategic Alignment This project aligns with the strategic goal of modernizing the Computer Science laboratory experience. By shifting to an automated DevOps-driven infrastructure, the institution minimizes the administrative burden on teaching assistants and admin, allowing them to focus on pedagogy rather than syntax checking. Furthermore, the platform introduces users to industry-standard workflows, preparing them for competitive programming environments and technical interviews.

Scope of Release 1.0 This SRS specifically covers the requirements for **Release 1.0**, which includes:

- User Authentication (JWT).
- The Problem Repository and Code Editor interface.
- The Docker-based Execution Service for Python and C languages.
- The CI/CD pipeline for automated system deployment.
- Basic analytics and Leaderboard functionality.

1.5 References

The following documents and web resources were used as references in the creation of this SRS or are necessary for understanding the underlying technologies used in the **Code-Judge** system.

Reference ID	Title / Description	Source / Link
REF-01	**IEEE Std 830-1998:** Recommended Practice for Software Requirements Specifications	IEEE Computer Society (1998)
REF-02	**RFC 7519:** JSON Web Token (JWT) Standard	https://tools.ietf.org/html/rfc7519
REF-03	**OpenAPI Specification (v3.0):** RESTful API Contract Standards	https://swagger.io/specification/
REF-04	**Docker Documentation:** Containerization and Sandboxing	https://docs.docker.com/
REF-05	**Kubernetes Documentation:** Orchestration and Pod Management	https://kubernetes.io/docs/home/
REF-06	**GitHub Actions Documentation:** CI/CD Pipeline Configuration	https://docs.github.com/en/actions
REF-07	**Prometheus & Grafana:** Monitoring Stack Implementation	https://prometheus.io/docs/introduction/overview/
REF-08	**Code-Judge System Architecture:** Microservices and Data Flow	Project Repository: `/docs/architecture/`
REF-09	**API Endpoint Definition:** Swagger UI Export	Project Repository: `/docs/api/swagger.json`

2. Overall Description

2.1 Product Perspective

Code-Judge is a new, self-contained software system designed to replace the manual and error-prone methods currently used for coding practice. Unlike monolithic predecessors, this product is built as a distributed system using a **Microservices Architecture**. This design ensures that the failure of one component (e.g., the grading engine) does not crash the entire application.

While the product functions independently, it relies heavily on an underlying container orchestration environment (Docker/Kubernetes) to perform its core function: the safe execution of untrusted code.

System Architecture & Components The system is divided into modular services that communicate via RESTful APIs through a central API Gateway. The high-level interaction between these components is illustrated below:

Component Interfaces

1. **User Interface (Frontend):**
    - The product provides a web-based interface (React/Vue) accessible via standard web browsers. It acts as the client, sending JSON requests to the API Gateway.
    - **Key Interaction:** The "Code Editor" component must handle real-time syntax highlighting and send raw code strings to the backend.
2. **API Gateway:**
    - Acts as the single entry point for all client requests. It handles routing, rate limiting, and request forwarding to the appropriate microservice.
3. **The Execution Engine (Interface with Docker):**
    - This is the critical interface between the Code-Judge software and the host infrastructure. The Execution Service communicates with the Docker Daemon (via Docker Socket or Kubernetes API) to dynamically create, monitor, and destroy containers.
    - **Constraint:** This interface must strictly enforce "Sandboxing" parameters (CPU limits, Network disabled) to ensure host security.
4. **Database Interface:**
    - Each microservice (Auth, Problem, Leaderboard) connects to its own logical data store (or shared database clusters like

PostgreSQL/MongoDB) to ensure data persistence for user profiles, problem sets, and submission history.

5. **CI/CD Pipeline (External Interface):**
    - The system integrates with **GitHub Actions** to automate the build and deployment process. When code is pushed to the repository, the pipeline builds new Docker images and deploys them to the active cluster.

Hardware and Software Constraints

- **Server Requirement:** The backend requires a Linux-based environment capable of running Docker or a Kubernetes cluster (Minikube for local/dev, Managed K8s for prod).
- **Client Requirement:** A modern web browser (Chrome, Firefox, Safari) with JavaScript enabled.
- **Network:** The system requires a stable internet connection for fetching libraries and checking user authentication, though the *execution* of code happens locally within the sandbox.

2.2 Product Features

The **Code-Judge** platform is functionally divided into four major modules: User Management, Problem Repository, the Execution Engine, and System Analytics. The system acts as a centralized hub connecting developers and the containerized grading infrastructure.

Feature Summaries

1. Secure Authentication & Role Management

- **Access Control:** The system utilizes JWT (JSON Web Tokens) to manage secure, stateless sessions.
- **Role-Based Access:**
    - **User:** Can view problems, submit solutions, and view their own history.
    - **Admins:** Have elevated privileges to create problems, upload hidden test cases (Input/Output files), and view class-wide analytics.

2. Problem Repository & Coding Interface

- **Discovery:** Users can browse the problem library, filtering by difficulty (Easy, Medium, Hard) or topic (e.g., Arrays, Dynamic Programming).

- **In-Browser Editor:** A dedicated coding environment (based on Monaco or similar) featuring syntax highlighting for supported languages (C).
- **Draft Execution:** Allows users to run code against "Sample Test Cases" to verify basic logic before a formal submission.

3. Automated Execution Engine (The "Sandbox")

- **Sandboxed Execution:** Upon submission, the system dynamically spins up an isolated Docker container. This ensures that user code runs in a secure, consistent environment without access to the host server.
- **Resource Throttling:** The engine strictly enforces CPU and Memory limits (e.g., 256MB RAM, 2s Timeout) to prevent infinite loops or system crashes.
- **Verdict Generation:** The system compares the code's standard output against expected "Hidden Test Cases" to generate a verdict: *Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE),* or *Compilation Error (CE).*

4. Analytics & Gamification

- **Real-Time Leaderboard:** A dynamic ranking system that updates immediately after a successful submission, awarding points based on problem difficulty.
- **Submission History:** A persistent log for every user, detailing past attempts, error logs, and performance metrics for self-review.

5. DevOps & Continuous Deployment

- **Automated Pipeline:** The system features a fully integrated CI/CD pipeline (GitHub Actions). Any code pushed to the repository is automatically linted, tested, and deployed to the Kubernetes/Docker cluster, ensuring high availability and rapid updates.

2.3 User Classes and Characteristics

The **Code-Judge** platform caters to three distinct user classes. These classes are differentiated by their motivation (practice vs. administration), technical expertise, and the specific functions they utilize within the system.

1. The Developer / Coding Enthusiast (Favored User Class)

This is the primary end-user of the system. This class includes software engineers preparing for technical interviews, developers practicing algorithms, or competitive programmers solving challenges for sport.

- **Frequency of Use:** Variable. Can range from "daily streaks" (high engagement) to sporadic usage (weekend contests).
- **Technical Expertise:** Varied (Novice to Expert).
    - *Novice:* Learning basic syntax and logic; requires helpful error messages.
    - *Expert:* optimizing for milliseconds and memory usage; requires precise execution metrics.
- **Privileges:** Standard User.
    - Can browse the problem library and filter by "Company Tags" (e.g., Google, Amazon) or Topic.
    - Can submit code and view their position on global leaderboards.
    - **Security Constraint:** Their code is untrusted and must be executed in a strict sandbox.
- **Key Requirements:** Instant feedback (low latency), a high-quality editor (IntelliSense/Autocomplete), and a reliable "Submission History" to track their improvement over time.

2. The Problem Setter / Content Moderator

This user class consists of senior developers or platform administrators responsible for expanding the problem library and ensuring the quality of challenges.

- **Frequency of Use:** Periodic. Usage is driven by the release schedule of new contest rounds or weekly challenges.
- **Technical Expertise:** High. They have a deep understanding of algorithmic complexity ($O(n)$ vs $O(n^2)$) and edge cases.
- **Privileges:** Administrative.
    - Can create new problems with rich descriptions and constraints.
    - Responsible for defining "Strong Test Cases" to prevent weak solutions from passing.
    - Can moderate discussion forums or solution walkthroughs.

- **Key Requirements:** Robust tools for uploading large test case files (Input/Output), ability to set custom time/memory limits per language, and analytics to see if a problem is "too hard" or "too easy."

3. The Platform Reliability Engineer (DevOps)

This user class is responsible for the uptime, scalability, and security of the public-facing platform.

- **Frequency of Use:** Constant (Monitoring) / As-needed (Deployment).
- **Technical Expertise:** Advanced Systems Knowledge. Proficient in Kubernetes scaling, Cloud infrastructure, and Security compliance.
- **Privileges:** Root / Infrastructure Access.
- Manages the auto-scaling groups to handle traffic spikes (e.g., during a "Weekly Contest").
- Monitors "Cost per Execution" (optimizing container efficiency).
- **Key Requirements:** High observability (Grafana Dashboards), automated CI/CD for zero-downtime deployments, and robust security alerts for attempted sandbox escapes.

2.4 Operating Environment

The **Code-Judge** system is a distributed application that operates across two distinct environments: the Client Side (User Interface) and the Server Side (Execution Infrastructure).

1. Client-Side Environment (The User) The client interface is designed to be lightweight and platform-independent, requiring only a standard web browser.

- **Hardware:** Any modern desktop or laptop computer. (Mobile/Tablet support is limited to "Read-Only" views for problem browsing, as coding on mobile is not a primary use case).
- **Operating System:** Platform independent (Windows, macOS, Linux).
- **Software:** A modern, JavaScript-enabled web browser. Supported versions include:
    - Google Chrome (v90+)
    - Mozilla Firefox (v88+)

  - Safari (v14+)
  - Microsoft Edge (Chromium-based)

2. Server-Side Environment (The Infrastructure) The backend requires a robust Linux ecosystem to manage the containerization and orchestration of the microservices.

- **Hardware Platform:**
  - **Development:** Local machine with at least 4-8GB RAM and 4 Cores (to support Minikube/Docker Desktop).
  - **Production:** Cloud Virtual Machines (e.g., AWS EC2, Google Compute Engine) capable of horizontal scaling. Recommended starting configuration: 16GB+ RAM to handle concurrent Docker container spawns.
- **Operating System:** Linux-based distribution (Ubuntu 20.04 LTS, CentOS 8, or Alpine Linux) is strictly required for the host nodes to run Docker natively and efficiently.
- **Container Engine:** Docker Engine (v20.10+)
- **Orchestration:** Kubernetes (v1.20+) or Docker Swarm.
  - Note: For local development, **Minikube** or **Kind** is used.

3. Software Dependencies & Coexistence The system relies on several key software components that must run simultaneously and communicate over the internal network:

- **Database Systems:** PostgreSQL (Relational data for Users/Problems) and MongoDB (Unstructured data for Submission Logs) running as persistent services.
- **Message Broker:** RabbitMQ or Kafka (optional, but often used in this architecture) to queue submission requests during high load.
- **Monitoring Stack:** Prometheus (Metrics scraper) and Grafana (Visualization dashboard) must run alongside the application services to monitor container health.
- **API Gateway:** Nginx or Traefik to handle ingress traffic and route requests to the appropriate microservice.

4. External Interfaces

- **CI/CD Environment:** The source code and deployment pipelines reside on **GitHub** and execute via **GitHub Actions** runners.

- **Network:** The server requires open outbound ports (HTTPS/443) to fetch updates/packages and inbound ports (80/443) for client traffic. The internal Docker network isolates the execution containers from the public internet.

2.5 Design and Implementation Constraints

These constraints outline the boundaries within which the development team must operate. They are derived from the necessity of running untrusted code safely and the architectural decisions made for scalability.

1. Architectural Constraints (Microservices & Docker)

- **Containerization is Mandatory:** All microservices and, critically, all user code executions **MUST** run inside Docker containers. Direct execution on the host OS is strictly prohibited to prevent security breaches.
- **Stateless Execution:** The Execution Service must be stateless. It cannot rely on local storage for persistence; once a grading container finishes, it is destroyed. All permanent data (results, logs) must be offloaded to the database immediately.
- **Orchestration:** The system must be deployable via **Kubernetes** (or Docker Swarm). The architecture must support horizontal scaling (adding more pods) rather than vertical scaling (making one server bigger) to handle concurrent submission spikes.

2. Hardware and Resource Limits (The Sandbox) To ensure fair usage and prevent system crashes, the "Sandboxed Containers" must adhere to strict hardware limitations defined in the Docker/Kubernetes config:

- **CPU Throttling:** Each submission container is limited to a specific CPU quota (e.g., 0.5 vCPU) to prevent a single "infinite loop" submission from freezing the node.
- **Memory Hard Limit:** Containers must be capped at 256MB - 512MB RAM. If a user's code exceeds this (Memory Limit Exceeded), the container must be killed immediately by the OOM (Out of Memory) Killer.
- **Network Isolation:** Submission containers must have **NO internet access**. They must run on an internal network with no outbound routes to prevent users from scraping external data or attacking third-party servers.

3. Technology Stack Constraints The development team is restricted to the following technologies to ensure compatibility and maintainability:

- **Backend Languages:** Node.js (Express) or Go (Golang) are preferred for the Execution Service due to their low overhead and fast startup times.
- **Database:**
  - **PostgreSQL:** For relational data (Users, Problems).
  - **MongoDB:** For unstructured logs and submission source code.
- **Communication Protocol:** Internal service-to-service communication must use **REST (HTTP/JSON)** or **gRPC**. Asynchronous tasks (like queuing a submission) should use a message broker (e.g., RabbitMQ).

4. Security and Regulatory Constraints

- **Root Privileges:** The application processes inside the containers must **NOT** run as root. User code must execute under a low-privilege user ID (UID > 1000).
- **Ephemeral Filesystems:** The file system for the submission container should be Read-Only, except for a temporary /tmp directory for compilation artifacts.
- **JWT Standards:** Authentication must use **RS256** (Asymmetric) or **HS256** signing for JSON Web Tokens. Tokens must have a short expiration time (e.g., 1 hour) with a refresh mechanism.

5. Development Pipeline Constraints

- **CI/CD Requirement:** No code can be deployed to production manually. All deployments must trigger through the **GitHub Actions** pipeline, which enforces linting and unit testing before building images.
- **Code Quality:** All microservices must adhere to standard linting rules (e.g., ESLint for JS, PEP8 for Python) before the build pipeline allows a merge.

6. Time Constraints (Latency)

Execution Overhead: The system introduces overhead by spinning up containers. The design must ensure that the "Boot Time" of a container is less than **2 seconds**. If the overhead is too high, the system will feel unresponsive to developers.

2.6 User Documentation

To ensure successful adoption and usability of the **Code-Judge** platform, the following documentation components will be delivered alongside the software release. These documents cater to the different technical levels of the user classes.

1. Online User Guide (Web-Based) Integrated directly into the application (accessible via a "Help" or "Docs" link in the navbar), this dynamic guide serves as the primary resource for end-users.

- **Target Audience:** Developers.
- **Key Contents:**
  - **"Getting Started":** A walkthrough of the registration, login, and dashboard interface.
  - **"Using the Editor":** Instructions on changing themes, resetting code, and using keyboard shortcuts.
  - **"Understanding Verdicts":** A detailed explanation of what each system response means (e.g., *Difference between "Runtime Error" and "Compilation Error"*, *Why "Time Limit Exceeded" happens*).
  - **"Environment Specifications":** A technical reference listing the exact compiler versions (e.g., GCC 9.3.0, Python 3.8) and available libraries for the sandboxed environment.

2. Administrator Manual (PDF & Wiki) A comprehensive guide designed for the "Problem Setters" who manage the content.

- **Target Audience:** Content Admins.
- **Key Contents:**
  - **Problem Creation Standard:** Detailed specs on how to format problem descriptions (Markdown support) and constraints.
  - **Test Case Formatting:** Instructions on how to structure input/output files (e.g., .in and .out files) and how to bulk-upload them (e.g., ZIP file structure).
  - **Analytics Dashboard:** How to generate and export class performance reports and plagiarism flags.
- **Format:** Delivered as a downloadable PDF and hosted as a Markdown Wiki on the project GitHub.

3. API Reference Documentation (Swagger/OpenAPI) Since the project is built on Microservices, the API documentation is crucial for developers or advanced users who wish to build plugins or integrations.

- **Target Audience:** DevOps / External Developers.
- **Key Contents:** Interactive documentation of all REST endpoints (POST /submit, GET /problems/{id}), including request bodies, response schemas, and authentication methods.

- **Delivery:** Auto-generated via Swagger UI and accessible at the /api-docs endpoint.

4. Video Tutorials Short, focused video clips embedded in the "Help" section.

- **Topics:** "How to Submit Your First Solution," "How to Create a Test Case," and "Troubleshooting Docker Errors (for Admins)."

5. Delivery Standards

- All written documentation will be provided in **English**.
- Documentation formats will adhere to standard **Markdown (.md)** for repository docs and **HTML5** for the web-based help center.
- The "Environment Specifications" page will be dynamic, pulling version numbers directly from the active Docker images to ensure accuracy.

2.7 Assumptions and Dependencies

This section lists the factors that are accepted as true for the purposes of development and the external elements upon which the project relies. If these assumptions prove incorrect or if dependencies fail, the project schedule or functionality may be compromised.

1. Assumptions

- **Container Startup Latency:** It is assumed that the host infrastructure (server) has sufficient I/O and CPU performance to spin up ephemeral Docker containers in under **3 seconds**. If container startup takes longer, the user experience (Real-time feedback) will be deemed unacceptable.
- **Test Case Validity:** It is assumed that the "Hidden Test Cases" provided by the Problem Setters are logically correct and strictly formatted. The system does not validate the correctness of the expected output; it only performs a string comparison.
- **Host OS Compatibility:** It is assumed that the deployment environment will be a **Linux-based** OS (Ubuntu/CentOS). While Docker runs on Windows/Mac, production-grade performance and security features (like OOM Killer and Cgroup strictness) are assumed to be managed via a Linux kernel.
- **Stable Internet for Deployment:** It is assumed that the deployment server has a stable internet connection to fetch base images (e.g., python:3.9-slim, gcc:latest) from Docker Hub during the build process.

- **Fair Usage:** It is assumed that while users may write inefficient code, they will not intentionally coordinate a Distributed Denial of Service (DDoS) attack against the execution cluster.

2. Dependencies

- **Docker Engine / Container Runtime:** The entire Execution Service is strictly dependent on the **Docker Daemon** (or a compatible runtime like Containerd) being active and accessible via the Unix Socket or API. If Docker crashes, the core feature of the app fails.
- **GitHub & GitHub Actions:** The CI/CD pipeline and source code management are dependent on GitHub's ecosystem. The project relies on **GitHub Actions Runners** to execute the build and test workflows.

Third-Party Libraries:

- **Monaco Editor:** The frontend relies on the Microsoft Monaco Editor library for syntax highlighting and code handling.
- **JWT Library:** The backend relies on standard libraries (e.g., jsonwebtoken for Node.js) for cryptographic signing and verification.
- **Language Compilers:** The system depends on the availability and stability of official compiler images (GCC, Python, OpenJDK) hosted on public registries.
- **Kubernetes (Minikube/K3s):** For the orchestration aspect, the system depends on a functioning Kubernetes cluster to manage pod scaling and networking.

3. System Features and Detailed requirements

1. User Management & Authentication

User Registration & Login: Users must be able to create accounts and log in securely using JWT (JSON Web Tokens).

Profile Management: Users should be able to view their submission history, total points, and ranking.

2. Problem Management

Problem Creation: Admins must be able to create problems with a title, description, constraints, and memory/time limits.

Test Case Management: Admins must be able to upload hidden test cases (Input files and corresponding Expected Output files).

Problem Categorization: Ability to tag problems by difficulty (Easy, Medium, Hard) and topic (e.g., "Recursion", "Strings").

3. Coding Workspace

Problem Discovery: Users must be able to browse and filter a list of available coding challenges.

Code Editor: A web-based editor with syntax highlighting for supported languages (C language).

Draft Execution (Run): Users must be able to run their code against "Sample Test Cases" before final submission.

Final Submission: A mechanism to trigger the full evaluation against hidden test cases.

4. Execution Engine (The Core)

Code Isolation (Sandboxing): The system must spin up a fresh, isolated Docker container for every execution to prevent malicious code from accessing the host server.

Compilation: use the appropriate compiler/interpreter version.

Resource Limitation: The system must enforce strict Time Limits (e.g., 2 seconds) and Memory Limits (e.g., 256MB) on the running container.

Verdict Generation: The system must compare the code output with the expected output and return one of the following:

- Accepted (AC)
- Wrong Answer (WA)
- Time Limit Exceeded (TLE)
- Memory Limit Exceeded (MLE)
- Runtime Error (RE)
- Compilation Error (CE)

5. DevOps & System Integrity

Automated Deployment: The system must trigger a CI/CD pipeline (GitHub Actions) to build and deploy microservices upon code changes.

Non Functional

1. Security & Isolation (Top Priority)

- **Sandboxing:** Every submission must run in a restricted, non-privileged Docker container with no network access to prevent users from attacking the host system or accessing other users' data.
- **Data Integrity:** Use **JWT (JSON Web Tokens)** for stateless, secure authentication to ensure students can only view or modify their own submissions.
- **Resource Throttling:** The system must prevent "Fork Bombs" or infinite loops by strictly limiting CPU shares and PIDs (Process IDs) within the container.

2. Scalability & Performance

- **Horizontal Scalability:** The Execution Service must be able to scale horizontally. During a peak exam hour, the system should spin up additional pods in Kubernetes to handle the increased load.
- **Low Latency:** The "Time to Verdict" should be minimized. The overhead of spinning up a Docker container should ideally be under **2-3 seconds**.
- **Concurrency:** The system must handle at least N (e.g., 50-100) simultaneous code submissions without crashing or significantly degrading performance.

3. Availability & Reliability

- **Fault Tolerance:** If the Execution Service fails during a run, the API Gateway should retry or redirect the request to a healthy instance without the student losing their progress.
- **Database Reliability:** Use persistent volumes for your databases (PostgreSQL/MongoDB) so that problem statements and submission histories are not lost if a container restarts.
- **Uptime:** The platform should aim for at least **99.9% uptime** during scheduled lab hours.

4. Observability & Maintainability

- **Monitoring:** Implement **Prometheus** to track metrics like the number of containers currently running and **Grafana** to visualize system health.
- **Logging:** Centralized logging is required to debug why a specific Docker container failed to compile a student's code.
- **CI/CD Efficiency:** The deployment pipeline should be fully automated, allowing a developer to push a bug fix and have it live in the cluster within minutes.

5. Usability & Portability

- **Cross-Browser Compatibility:** The frontend (Monaco Editor/React) must function correctly on Chrome, Firefox, and Safari.
- **Platform Independence:** Because the system is containerized, it must be able to run on any environment (Local, AWS, Azure, or Google Cloud) without code changes.

Requirement ID	Title/Feature	Description	Type / Category	Priority
FR-AUTH-01	**User Registration**	The system must allow users to register using an email and password, storing the password as a hashed value (e.g., bcrypt).	Functional	**High**
FR-AUTH-02	**JWT Authentication**	The system must issue a JSON Web Token (JWT) upon login to manage stateless sessions for API access.	Functional	**High**
FR-PROB-01	**Problem Browsing**	Users must be able to view a paginated list of problems filtered by Difficulty (Easy/Med/Hard) and Topic.	Functional	**High**

FR-PROB-02	**Create Problem (Admin)**	Admins must be able to create new problems with Markdown descriptions and specific constraints (Time/Memory limits).	Functional	**High**
FR-PROB-03	**Test Case Upload**	The system must allow Admins to upload hidden input.txt and expected output.txt files for problem validation.	Functional	**High**
FR-EDIT-01	**Code Editor**	The interface must provide a code editor (Monaco/Ace) with syntax highlighting for supported languages (C, Python).	Functional	**High**

FR-EXEC-01	**Docker Sandbox**	Upon submission, the system must spin up an isolated, ephemeral Docker container to execute the user's code.	Functional	**High**
FR-EXEC-02	**Verdict Generation**	The system must compare the container's standard output (stdout) with the expected output and return a	Functional	**High**

		verdict (AC, WA, CE, RE).		
FR-EXEC-03	**Time Limit Enforcement**	The Execution Engine must strictly enforce the defined time limit (e.g., 2.0s). If exceeded, it must kill the container and return "TLE".	Functional	**High**
FR-EXEC-04	**Language Support**	The system must support compilation and execution for at least C (GCC) and Python 3.	Functional	**Medium**
FR-RANK-01	**Real-time Leaderboard**	The system must update and display global user rankings based on total points immediately after a successful submission.	Functional	**Medium**

FR-DEVOPS-01	**CI/CD Pipeline**	The system must utilize GitHub Actions to automatically lint, test, and deploy microservices upon a code push.	Functional	**Medium**
NFR-SEC-01	**Network Isolation**	Submission containers must have **Network Access Disabled** (no internet) to prevent external attacks or data exfiltration.	Non-Functional (Security)	**High**
NFR-SEC-02	**Privilege Escalation**	User code must execute as a non-root user (UID > 1000) inside the container to prevent host OS tampering.	Non-Functional (Security)	**High**
NFR-PERF-01	**Container Latency**	The overhead for spinning up a fresh Docker container must be less than **2 seconds** per submission.	Non-Functional (Performance)	**High**

NFR-PERF-02	**Concurrency**	The system must handle at least 50 simultaneous code submissions without service degradation.	Non-Functional (Scalability)	**High**
NFR-REL-01	**Fault Tolerance**	If a grading container crashes (Runtime Error), the system must handle the exception gracefully and report "Runtime Error" rather than crashing the backend.	Non-Functional (Reliability)	**High**
NFR-USAB-01	**Browser Compatibility**	The web interface must be fully functional on Chrome, Firefox, and Safari (desktop versions).	Non-Functional (Usability)	**Medium**

NFR-MAINT-01	**Centralized Logging**	The system must aggregate logs from all microservices (Auth, Execution, API Gateway) into a central dashboard for debugging.	Non-Functional (Maintainability)	**Low**

4. External Interface Requirements

4.1 User Interfaces

The user interface (UI) serves as the visual entry point to the microservices backend. It is designed as a Single Page Application (SPA) to ensure a seamless, desktop-application-like experience. The interface prioritizes clarity and minimal distraction, adhering to modern "Developer Experience" (DX) standards.

4.1.1 GUI Standards and Style Guidelines

- **Theme:** The application shall default to **Dark Mode** (High Contrast) to reduce eye strain during long coding sessions, consistent with industry-standard IDEs (e.g., VS Code).
- **Typography:**
  - **Code Blocks:** Must use a Monospaced font (e.g., Fira Code, Consolas) to ensure alignment of characters.
  - **UI Text:** Sans-Serif font (e.g., Inter, Roboto) for readability.
- **Responsiveness:** The layout must be fluid using a grid system (e.g., CSS Grid/Flexbox). While coding is optimized for Desktop (1024px+), the dashboard and problem browsing must be usable on Mobile/Tablet devices.
- **Verdicts:** Color coding is strictly enforced for grading results:
  - **Green (#4CAF50):** Accepted (AC)
  - **Red (#F44336):** Wrong Answer (WA) / Runtime Error (RE)

- **Yellow/Orange (#FF9800):** Time Limit Exceeded (TLE) / Compilation Error (CE)

4.1.2 Main Screen Layouts

1. The Coding Workspace (Primary Interface) This is the most critical screen where users spend 90% of their time. It follows a "Three-Pane Layout":

- **Left Pane (Problem Description):** Displays the problem title, description (rendered Markdown), constraints, and sample Input/Output tables.
- **Right Pane (Code Editor):** An embedded instance of the Monaco Editor (VS Code engine) supporting syntax highlighting, line numbers, and auto-indentation.
- **Bottom Pane (Console/Terminal):** A collapsible section that displays the output of "Run Code" commands, compiler errors, and the final verdict.

2. The Problem Dashboard A list view allowing users to browse challenges.

- **Table Columns:** Status (Solved/Unsolved Icon), Title, Difficulty (Tag), Topic, and Acceptance Rate.
- **Filters:** Sidebar or Dropdown to filter by "Difficulty: Hard" or "Tag: Dynamic Programming."

3. Admin Problem Creation Wizard A form-based interface for Faculty/Admins.

- **Rich Text Editor:** For writing problem descriptions.
- **File Uploader:** A "Drag and Drop" zone for uploading bulk test cases (Zipped .in and .out files).

4.1.3 Standard Buttons and Navigation

- **Global Navigation Bar:** Fixed at the top of every screen. Contains:
  - *Left:* Logo (Home), Problems, Leaderboard.
  - *Right:* User Profile Avatar, Notification Bell, Logout.
- **Action Buttons (Workspace):**
  - **"Run Code" (Secondary Action):** executes code against sample test cases.
  - **"Submit" (Primary Action):** executes code against hidden test cases. Prominently placed (usually top-right or bottom-right).

4.1.4 Error Message Display Standards The system distinguishes between "System Errors" and "User Errors."

- **System Errors (e.g., Network Fail, Server 500):** Displayed as a **Toast Notification** (Popup) at the top-right corner. *Example: "Connection lost. Reconnecting..."*
- **User/Logic Errors (e.g., Compilation Error):** Displayed specifically in the **Console Pane**. The error log from the compiler (gcc/python) must be preserved and shown in raw text to help the user debug.

4.1.5 Keyboard Shortcuts To enhance developer productivity, the web interface must support standard IDE shortcuts:

- Ctrl + Enter (or Cmd + Enter): Trigger "Run Code".
- Ctrl + S (or Cmd + S): Trigger "Save Draft" (Local Storage).
- Tab: Insert 4 spaces (or 2 spaces) depending on language config.

4.2 Hardware Interfaces

Since **Code-Judge** is a web-based distributed application, it does not control custom hardware (like sensors or robotics). However, the **Execution Service** has a critical logical interface with the host server's hardware resources (CPU, RAM, Storage) to enforce the "Sandboxing" security model.

4.2.1 Server-Side Resource Management (Logical Interface) The software interacts with the physical server hardware through the Operating System kernel and the Container Runtime (Docker Engine).

- **Processor (CPU) Interaction:**
  - **Logical Control:** The Execution Service utilizes **Linux Control Groups (cgroups)** to strictly limit the number of CPU cycles a submission container can consume.
  - **Interaction:** The software sends a constraint (e.g., --cpus=".5") to the Docker Daemon, which translates this into kernel scheduling instructions for the physical CPU cores.
- **Memory (RAM) Interaction:**
  - **Logical Control:** The system interfaces with the host's memory management unit (MMU) to enforce a hard cap (e.g., 256MB).
  - **OOM Killer:** If a user's code attempts to address physical memory beyond this limit, the hardware/OS interface triggers an "Out of Memory" signal, which the software captures and reports as Memory Limit Exceeded.

- **Storage (Disk) Interaction:**
  - **Ephemeral Storage:** The software creates temporary, Copy-on-Write (CoW) layers on the physical disk for every submission container. These are physically destroyed immediately after execution to free up inodes.
  - **Persistent Storage:** The Database microservices mount physical disk partitions (Volume Mounting) to store user data and problem sets permanently.

4.2.2 Client-Side Device Support

- **Input Devices:**
  - **Keyboard:** The web interface is optimized for standard **HID (Human Interface Device)** keyboards. It listens for standard ASCII input and specific key-chord events (e.g., Ctrl+Enter) via the browser's event API.
- **Display Devices:**
  - **Resolution:** The UI is designed for a minimum physical resolution of **1366x768 (HD)**, but optimized for **1920x1080 (FHD)**.
  - **Constraint:** Screen real estate is required to render the "Three-Pane Layout" (Description, Editor, Terminal) simultaneously without overlapping.

4.2.3 Network Hardware Interface

Network Interface Cards (NIC): The software interacts with the server's physical NIC via the TCP/IP stack.

4.2.4 Communication:

Ingress: Port 443 (HTTPS) and Port 80 (HTTP) must be logically open on the firewall to accept traffic.

Internal: The Microservices communicate over a virtualized bridge network (Docker Network / Kubernetes CNI), which abstracts the physical routing layer.

4.3 Software Interfaces

The **Code-Judge** system relies on a network of interconnected software components. This section defines the specific APIs, database connections, and libraries that facilitate data exchange between the modules.

4.3.1 Operating System & Container Runtime

- **Host OS:** The system is designed to run on **Linux** (Ubuntu 20.04 LTS or Alpine Linux 3.14+). It relies on the Linux Kernel features (Namespaces, Cgroups) for isolation.
- **Docker Engine (v20.10+):** The Execution Service communicates directly with the Docker Daemon via the **Docker Engine API** (Unix Socket or HTTP).
    - **Incoming Message:** POST /containers/create (Config: Image, Cmd, HostConfig).
    - **Outgoing Message:** Container ID, Log Stream (stdout/stderr), Exit Code.
    - **Purpose:** To lifecycle manage the ephemeral sandboxes.

4.3.2 Database Interfaces The microservices connect to dedicated data stores via standard TCP/IP drivers.

- **PostgreSQL (v13+):** Used by the **Auth Service** and **Problem Service**.
    - **Interface:** Accessed via an ORM (e.g., Sequelize/TypeORM) or native driver.
    - **Data Shared:** User Credentials (Hashed), User Roles, Problem Descriptions, Test Case Paths (Metadata).
- **MongoDB (v5.0+):** Used by the **Execution Service** and **Leaderboard Service**.
    - **Interface:** Accessed via Mongoose or native driver.
    - **Data Shared:** Raw Submission Code, Execution Logs, Verdict History (JSON documents).
    - **Purpose:** Storing unstructured, high-volume log data that doesn't fit a rigid schema.

4.3.3 External APIs & Libraries

- **GitHub Actions (CI/CD):**
    - **Interface:** Webhooks.
    - **Direction: Incoming**.
    - **Message:** push event payload (Branch name, Commit Hash).
    - **Purpose:** Triggers the automated build and deployment pipeline whenever code is committed to the repository.
- **Monaco Editor (v0.25+):**
    - **Interface:** JavaScript Library (Frontend).
    - **Purpose:** Provides the in-browser coding experience (IntelliSense, Syntax Highlighting).

- **JSON Web Tokens (JWT):**
  - **Standard:** RFC 7519.
  - **Data Item:** The Authorization header in every HTTP request containing the Bearer Token.
  - **Purpose:** Stateless identity verification across all microservices.

4.3.4 Internal Microservice Communication Services communicate internally over a private network (Kubernetes Cluster IP) using **RESTful APIs** over HTTP/1.1.

Source Service	Destination Service	Protocol	Data Transmitted	Purpose
API Gateway	**Auth Service**	HTTP/POST	Login Credentials	Validate user and issue Token.
API Gateway	**Execution Service**	HTTP/POST	Source Code, Language, ProblemID	trigger a grading run.
Execution Service	**Problem Service**	HTTP/GET	ProblemID	Fetch Hidden Test Cases (Inputs) for grading.
Execution Service	**Leaderboard Service**	HTTP/POST	UserID, Points, Verdict	Update global rankings after a successful run.

4.3.5 Implementation Constraints regarding Data Sharing

- **No Shared Memory:** Because the system is distributed across containers, services **cannot** share global variables or memory addresses. All data sharing must occur via **Network Calls (API)** or **Database Persistence**.
- **Test Case Security:** The "Hidden Test Cases" (Input/Output files) stored in the Problem Service must **never** be exposed to the Frontend API. They are exclusively for internal use by the Execution Service.

4.4 Communications Interfaces

This section defines the network protocols and communication standards required for the system to function securely and efficiently across the public internet and within the private cloud infrastructure.

4.4.1 Network Protocols

- **Hypertext Transfer Protocol Secure (HTTPS):** All external communication between the Client (User's Web Browser) and the API Gateway must occur over **HTTPS (Port 443)**. Unencrypted HTTP traffic (Port 80) must be automatically redirected to HTTPS.
- **WebSocket Protocol (WSS):** To provide real-time feedback without page refreshes, the system utilizes secure WebSockets (wss://). This allows the Execution Service to push "Live Logs" (e.g., Compiling..., Running Test Case 1...) directly to the user's browser as they happen.
- **SMTP (Simple Mail Transfer Protocol):** The Auth Service interfaces with an external email provider (e.g., SendGrid/AWS SES) via SMTP to send transactional emails (Password Reset links, Welcome emails).

4.4.2 Message Formatting & Serialization

- **JSON (JavaScript Object Notation):** This is the strict standard for all API request and response bodies.
    - **Content-Type:** All REST endpoints must consume and produce application/json.
    - **Date Format:** All timestamps (e.g., submissionTimestamp) must be formatted as **ISO 8601 Strings** (e.g., 2023-10-27T10:00:00Z) to ensure

timezone consistency between the server (UTC) and the user's local browser.

- **Multipart/Form-Data:** Used exclusively for the **File Upload** interface when Admins upload bulk test cases (ZIP files) or images.

4.4.3 Communication Security

- **TLS 1.2+ Encryption:** All data in transit (between the user and the server) must be encrypted using Transport Layer Security (TLS) version 1.2 or higher.
- **CORS (Cross-Origin Resource Sharing):** The API Gateway must implement strict CORS policies. It should only accept requests from the whitelisted frontend domain (e.g., https://code-judge.university.edu) to prevent malicious third-party sites from triggering unauthorized code executions.

4.4.4 Synchronization & Asynchronous Patterns

- Since code execution is a blocking operation (it takes time to compile and run), the communication interface follows an **Asynchronous Polling** or **Event-Driven** pattern:
- **Request:** The Client POSTs code $\rightarrow$ Server returns 202 Accepted and a submissionID.
- **Processing:** The Client maintains an open connection (WebSocket) or polls the server every 2 seconds (GET /submission/{id}) to check the status (PENDING $\rightarrow$ PROCESSING $\rightarrow$ COMPLETED).
- **Completion:** Once the verdict is ready, the server sends the final JSON payload, and the client updates the UI.

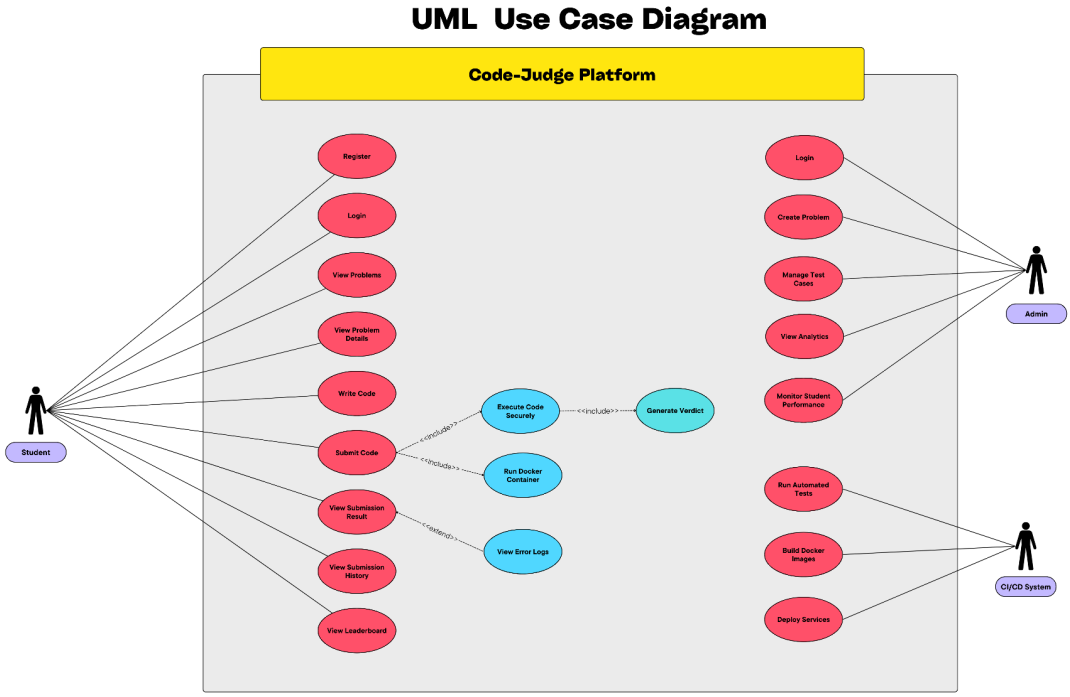
Appendix A: Glossary

Term / Acronym	*Definition*
Code-Judge	*The automated code evaluation platform being specified in this document.*
Sandbox	*A security mechanism for separating running programs; in this system, it refers to isolated Docker containers used to run untrusted user code.*
Docker	*The containerization platform used to create consistent, isolated execution environments for code submissions.*
Verdict	*The final status returned by the system after evaluating a submission (e.g., Accepted, Wrong Answer, TLE).*
JWT (JSON Web Token)	*A compact, URL-safe means of representing claims to be transferred between two parties, used here for secure authentication.*
AC (Accepted)	*A verdict indicating the user's code produced the correct output for all test cases.*
WA (Wrong Answer)	*A verdict indicating the user's code produced output that does not match the expected hidden test case.*

TLE (Time Limit Exceeded)	*A verdict indicating the user's code ran longer than the maximum allowed duration (e.g., 2 seconds).*
MLE (Memory Limit Exceeded)	*A verdict indicating the code attempted to use more RAM than the allocated limit (e.g., 256MB).*
CE (Compilation Error)	*A verdict indicating the source code could not be successfully compiled by the language-specific compiler (e.g., GCC).*
CI/CD	***Continuous Integration / Continuous Deployment** – The automated process of building, testing, and deploying system updates via GitHub Actions.*
Microservices	*An architectural style that structures the application as a collection of small, independent services (Auth, Problem, Execution).*
API Gateway	*The single entry point that routes client requests to the appropriate backend microservices.*
Test Case	*A pair consisting of an "Input" file and an "Expected Output" file used to validate the logic of a user's code.*
MoSCoW	*A prioritization technique (**M**ust have, **S**hould have, **C**ould have, **W**on't have) used to categorize project requirements.*
Kubernetes (K8s)	*An open-source system for automating deployment, scaling, and management of containerized applications.*

OOM Killer	***Out Of Memory Killer*** *– A Linux kernel process that terminates a container if it exceeds its hardware memory limits.*

Appendix B: Analysis Models



Appendix C: Issues List

Issue ID	***Description***	***Status***	***Priority***
ISS-01	***Docker Startup Latency:*** *Investigating if cold-start of containers exceeds the 3-second responsiveness goal.*	*Open*	*High*

ISS-02	***Plagiarism Detection:*** *Decision pending on whether to integrate MOSS or build a custom string-matching service.*	*TBD*	*Medium*
ISS-03	***Hardware Scaling:*** *Determination of exact AWS instance types required for concurrent handling of 100+ users.*	*Pending*	*High*
ISS-04	***Multi-Language Support:*** *Finalizing the list of supported C compiler flags (e.g., -O2, -Wall) for standardized grading.*	*In Review*	*Medium*
ISS-05	***SSO Integration:*** *Waiting for University IT API documentation to confirm compatibility with current Auth service.*	*Blocked*	*Low*
ISS-06	***Database Redundancy:*** *Conflict resolution required on whether to use a managed RDS or self-hosted PostgreSQL cluster.*	*Open*	*Medium*